

AgentFlame: 面向 AI 编程 Agent 系统效应的语义归因剖析

AgentSight 研究原型

2026 年 6 月 15 日

摘要

AI 编程 agent 的可观测性问题不只是“模型调用了哪个 tool”，而是用户语义意图如何导致真实系统副作用：进程、文件、网络、测试、失败重试和 token 消耗。现有 LLM observability 工具擅长展示单次运行的 trace tree、span duration、tool call 和 cost；传统 profiler 和进程工具擅长聚合系统行为。二者分开时，一个用户常问的问题仍然难答：哪个任务语义造成了哪些重复、沉重或越界的系统足迹？本文提出 AgentFlame，一种语义系统效应剖析模型：本地小模型只给 session、prompt 和 LLM call 生成一个词标签；当前 full run 从 agent-native tool logs 继承这些标签，目标模式则让 tool、shell、child process 与 file/network effect 通过确定性 lineage 继承这些标签；最终以 folded stack 形式聚合为 flamegraph。在本机 AgentSight 仓库的 205 个可读 Codex/Claude 真实 session 上，AgentFlame 标注 2,463 个 prompt 和 90,930 个 LLM call，0 个最终标签失败；它把 167,005 个系统观测折叠成 24,295 条语义系统栈。去掉 session/prompt 语义后，nonsemantic baseline 有 90.219% 的观测权重落入混合多个语义区域的 buckets；flat effect baseline 的混合权重为 90.770%。R131 消融进一步显示：在保持同一 167,005 个系统观测总权重不变时，session-only 仍混合 84.180% 的完整语义权重，prompt-only 降到 37.687%，完整 session+prompt 降到 0.000%。这些结果支持一个机制性结论：语义帧，尤其是 prompt 标签，能分开传统 trace 或 process summary 会合并的 task-effect 区域。本文也明确边界：当前 full run 仍基于 agent-native session 历史；R114 在 20 个固定 codex command-mode 任务上把 1,273/1,273 个 in-scope effects 连到 agent process family，precision/recall 均为 100.0%，并拒绝 3,170 个 observed negative-control effects；但用户任务实验、跨仓库规模和 full-history exact integration 仍未完成，因此不能声称用户效用或完整系统 provenance 已被证明。

1 问题

一个 AI 编程 agent 运行结束后，开发者通常不想逐条回放消息。他们想知道：这次运行哪里最重，哪里绕路，哪里反复试错，哪些文件或网络目标被碰过，哪类用户请求导致最多测试、最多读取、最多失败或最多 token。传统工具各自只覆盖一半。LLM trace 工具能展示 agent、LLM call、tool call、prompt、completion、latency 和 cost。进程或文件审计工具能展示 git、cargo、rg、sed、docker 等系统行为。Span-duration flamegraph 甚至已经出现在 multi-agent workflow debugging 中，用于看 critical path、parallelism、error cascade 和 tool overhead。

但是这些视图仍难回答跨层问题：

为了哪个用户任务，agent 反复运行了哪些命令、读取了哪些路径、产生了哪些网络或文件副作用？

本文的核心判断是：agent observability 的对象不应只是 span，也不应只是进程，而应是 *task-grounded system effect*。我们把目标栈写成：

```
sessionTag;promptTag;llmcall/tool;process*;effect
```

这个模型把两类传统工具分开的信息合并起来。上层的 session、prompt、LLM call 用本地小模型压缩成一个词；下层 tool、process、file/network effect 不由模型猜测，而通过确定性关联继承语义上下文。最后用 folded stack 聚合重复路径，让宽度表示 event count、effect weight 或 token weight，而不是只表示 span duration。

本文的贡献不应表述为“agent flamegraph”。已有系统已经把 agent spans 画成 flamegraph。本文的贡献是一个更具体的系统归因模型：在 agent-native local histories 中把用户语义意图与系统行为代理接起来，并为 live AgentSight traces 定义 exact provenance 目标，使跨 session 的重复副作用可聚合、可检查、可比较。

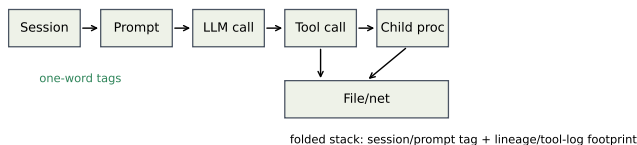


图 1: AgentFlame 的归因模型。小模型只标注 session、prompt、LLM-call 语义；tool/process/file/network effects 通过确定性 lineage 继承语义帧并折叠成 stacks。

2 设计

图 1 展示了 AgentFlame 的设计。语义层非常小：每个 session、prompt 和 LLM call 只生成一个 lowercase word，例如 `refactor`、`review`、`test`、`research`。这些词不是固定 ontology，也不是 verdict；它们只是给折叠栈提供短、稳定、可搜索的任务帧。

系统层不交给 LLM 判断。Agent-native 历史模式从 Codex/Claude JSONL 中恢复 tool 类型、命令入口、状态、路径组和粗 effect 类别。AgentSight exact 模式的目标输入是：

```
tool_call -> shell -> child process ->
file/network effect
```

每个 in-scope effect 必须能追溯到 tool call 和 prompt ancestry。如果只能用 PID 或时间戳模糊匹配，就不能作为强 provenance claim。因此，语义由上层继承，系统事实由 instrumentation 或 session parser 产生。

2.1 为什么是一个词

一个词标签有三个工程优点。第一，火焰图帧必须短；长摘要会让图不可读。第二，标签只用于导航和聚合，不承担安全、正确性或因果判决。第三，一个词标签容易本地化和缓存化。当前实现对 session/prompt/LLM 文本各处理一次，后续数十万条系统事件只做普通程序关联和聚合。

这也约束了论文 claim。AgentFlame 不证明标签是真实意图 ontology。它先证明一个更可审计的机制：加入这些短语义帧后，传统 baseline 会合并掉的系统效应是否被分开。

2.2 Stack 语法

系统效应栈形如：

```
project;agent;session;prompt;call:tool/...;process*;effect;
```

Token 栈形如：

```
project;agent;session;prompt;call:llm/...;model;kind
```

完整视图保留 session、prompt、LLM-call 语义。消融视图删除部分语义轴：`session-system`、`prompt-system`、`llm-token` 等。所有视图共享同一事实表；投影不应改变总权重。

3 实现

AgentFlame 现在是独立 Rust CLI，位于仓库 `agentflame/`。它使用 `normalize-chat-sessions` 读取本地 Codex 和 Claude JSONL 历史，调用 `llama.cpp-compatible` HTTP server 生成标签，输出 `agentflame.json`、`tags.json`、folded stack 文件、SVG 和 HTML dashboard。默认输出路径是：

```
.agentsight/agentflame/latest/
```

本次 full run 的命令为：

```
cargo run --manifest-path
agentflame/Cargo.toml -- run --project-root
. --scan-files 10000 --max-sessions
10000 --llama-url http://127.0.0.1:18080
--model local --timeout 60 --out
.agentsight/agentflame/latest
```

模型为本地 `qwen2.5-3b-instruct-q4_k_m.gguf`，通过 `llama.cpp` server 运行，temperature 为 0，并用 grammar 约束输出一个词。AgentFlame 没有 heuristic fallback：如果 LLM server 缺失，或模型重试后仍不能输出合法一词标签，run 失败。

3.1 隐私与可复现

原始 agent trace 不提交。`agentflame.json` 默认包含 redacted previews、hash、tag、计数和路径组；`tags.json` 保存 tag cache 和 LLM provenance，不保存原始 prompt 文本。本次 run 遇到一个 root-owned Claude JSONL，不可读；系统没有要求 root 权限读取，而是跳过并写入 `warnings`。这让覆盖范围可审计，而不是静默假装完整。

3.2 与 AgentSight 的关系

AgentFlame 的第一模式是零 instrumentation 的历史分析。AgentSight 的特点是运行时 exact provenance: `tool_call -> shell -> child process -> file/network effect`。正确集成后, AgentFlame 负责语义帧, AgentSight 负责系统效应事实, 两者通过 `tool/session/prompt ancestry` 连接。当前 full run 还不是 live exact-effect 结果。R110 增加了一个 live in-scope smoke; R111 把最小 session/tool envelope 移入 native collector report export; R112 把该 envelope 显式 backfill 到 SQLite sessions 和 tool_calls 表, 并用 `--no-observed-projection` 验证 persisted-only export; R113 在 `record -- <command>` 启动目标进程时写入 capture-time session/tool rows; R114 用 20 个真实 Codex command-mode 任务验证该路径, 加入 per-task negative controls, 并在 scoped oracle 下达到 1,273/1,273 in-scope effects joined、0 false positives、0 false negatives。C4 因此可支持固定 command-mode suite, 但仍不是 full-history 或跨仓库 benchmark。

4 评估

4.1 RQ1: 本地小模型能否全量标注?

表 1 给出 full run 结果。AgentFlame 分析了 205 个可读 session, 其中 Codex 78 个、Claude 50 个、Claude subagent 77 个。它处理 2,463 个 prompt rows 和 90,930 个 LLM-call events。标签器共有 93,598 个请求, 64,297 个命中 cache, 29,302 次真实 llama.cpp HTTP calls, 0 个最终失败。Prompt 和 LLM-call tag 的非法计数均为 0。这说明一词语法和本地 3B 模型路径在真实历史规模上可运行。

这个结果不等于标签语义充分。例如某些 tag 明显带噪声: `agentsightsm`、`testcodex`、`bashoutput`。因此 RQ1 支持的是 feasibility 和 syntax claim; tag adequacy 需要单独实验。

4.2 RQ2: 语义帧是否分割 baseline 混合?

Full run 产生 167,005 个 system observations, 并折叠成 24,295 条 unique semantic system stacks, 压缩率为 6.874 倍。关键不是压缩本身, 而是 baseline mixing。当删除 session/prompt 语义帧后, nonsemantic folded baseline 有 4,209 个 mixed buckets, 覆盖 150,670 个观测, 也就

表 1: 全量本机 AgentSight session 运行指标。

指标	数值
Readable sessions	205
Codex / Claude / Claude-subagent	78 / 50 / 77
Raw tool events	130,632
Raw LLM events	90,930
Prompt rows	2,463
Unique prompt tags	303
LLM-call tags	90,930
Unique LLM-call tags	1,250
Tag requests	93,598
Cache hits	64,297
llama.cpp HTTP calls	29,302
Final tag failures	0

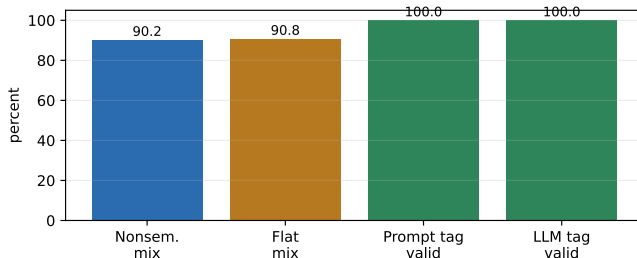


图 2: 当前机制结果。图由 `docs/visexp/paper/make_figures.py` 从 `.agentsight/agentflame/latest/agentflame.json` 重新生成; 它反映 Rust full-run 输出, 而不是旧的 Python sampled-run artifact。

是 90.219% 的系统权重。如果进一步退化成 flat effect baseline, mixed weight 为 151,590, 占 90.770%。

这为回应“传统工具也能看出来”的质疑提供了机制性证据。传统 flat summary 可以告诉我们 `sed read` 有 25,755 次、`rg read` 有 15,336 次、`cargo test` 有 2,903 次。但是它不知道这些行为分别来自 `refactor`、`review`、`research`、`design` 还是 `test` prompt 区域。AgentFlame 的价值是把 heavy/repeated system effects 按任务语义拆开。

R131 把这个结论进一步做成轴级别消融。每个 variant 都从同一个 folded input 投影而来, 并检查总权重保持 167,005 不变。同时, R131 记录 `agentflame.json` totals 与 folded inputs 匹配, 并验证已有 nonsemantic/session/prompt folded files 与 projection exact match。表 3 显示, system-effect 分割主要来自 prompt 轴: session-only 只把 mixed full-semantic bucket weight 从

表 2: 语义栈与 baseline mixing。

指标	数值
System observations	167,005
Unique semantic system stacks	24,295
Semantic compression	6.874x
Max stack reuse	6,004
Nonsemantic mixed buckets	4,209
Nonsemantic mixed weight	90.219%
Flat mixed buckets	4,051
Flat mixed weight	90.770%

表 3: R131 system-effect 语义轴消融。所有行都保持 167,005 总权重。

Variant	Unique stacks	Bucket	Residual
No semantic	10,641	90.219%	44.639%
Session only	13,328	84.180%	34.138%
Prompt only	22,341	37.687%	7.526%
Session + prompt	24,295	0.000%	0.000%

90.219% 降到 84.180%，prompt-only 则降到 37.687%。如果只计算 mixed buckets 中非主导 full semantic variants 的 residual weight，prompt-only 也从 no-semantic 的 44.639% 降到 7.526%。完整 session+prompt 视图的 0.000% mixed weight 是定义上的上界，不应被解释为用户效用；它只说明 full semantic key 没有在投影中被合并。

4.3 Case Study: 能看出什么

本次 full run 中，最大的 semantic stack 是：

```
project:agentsight;agent:codex;session:refactor;prompt:refactor;call:tool/tool;effect:process;status:ok
```

权重为 6,004。另一个明确的系统例子是 cargo test。Flat summary 只会显示 2,903 次 successful cargo test。Nonsemantic stack 会保留 call:tool/shell;process:cargo;effect:test;status:ok，但仍把不同语义区域混在一起。Semantic stack 将它拆成多个区域，包括 review/review、refactor/refactor、research/explain、research/refactor、design/review 等。

这类结果生成可供检查的候选分解，用来辅助三类 developer questions：

- 哪个任务导致最多重复测试或读取？

- 哪些 prompt tag 反复触碰同一组路径或域名？
- 哪些失败行为集中在特定 semantic region，而不是全局系统问题？

Trace tree 能回放某次调用，span flamegraph 能看 duration bottleneck，flat summary 能看重命令。但它们都不直接给出跨 session 的 task-effect 聚合。

4.4 RQ3: exact lineage 是否成立？

当前答案是：固定 command-mode suite 已经成立，但 broad full-history provenance 仍不足。effect_lineage_smoke.py 先在 AgentSight-shaped fixture 上验证 process/file/network effects 可以连到 session/tool/prompt ancestry。R110–R112 把同一思想推进到 3 个真实 AgentSight SQLite exports：先由 harness 增加 agent-run envelope，再移入 native export，最后写入 DB 副本；这些旧 snapshots 只能 join 182/318 raw effects，说明 envelope 存在但覆盖还不够。R113 把 record -- <command> 的 agent-run envelope 放到 capture path；R113-live 随后在 5 个真实只读 codex exec 任务上验证该路径能创建 5/5 sessions/tools 并 join 508/508 raw effects。R114 将这个路径扩展到 20 个固定 Codex tasks，包括 read-only、edit、test/debug、dependency、failure/retry 和 disposable-workspace write tasks，并加入 per-task negative-control bursts。在 scoped oracle 下，R114 结果为：20/20 target tasks completed, 20/20 tasks observed negative controls, 1,273/1,273 in-scope effects joined, 0 false positives, 0 false negatives, precision 和 recall 都是 100.0%，3,170 个 observed negative-control effects 中 0 个被 join。Raw join 仍只有 22.055%，这是预期结果：wrapper、sibling 和 out-of-scope effects 应该保持 on-plan，而不是被错误归因给 agent。

这个结果已经支持“固定 command-mode suite 中 exact semantic-effect lineage 成立”。它仍不能支持“任意历史、任意仓库、任意 agent 都有完整 provenance”。完整 OSDI artifact 还需要跨仓库 replication、更多 agent 类型、network-effect cases、child-depth/path/domain breakdown，以及用户任务结果。

4.5 RQ4: 用户效用是否成立？

当前答案是不成立，或者更准确地说：未验证。已有 task packet、answer key 和 scorer 原型，但没有真实参与者响应。下一步需要把 baseline 设为 trace tree、span

duration flamegraph、flat process summary、nonsemantic folded stack 和 semantic folded stack，比较 accuracy、time、false positives 和 confidence。如果这个实验失败，AgentFlame 仍可能是专家 forensic 工具，但不能写成提升普通开发者效率。

4.6 RQ5: 小模型成本和标签质量

Full run 和 R123 证明 3B 模型路径可用，但没有证明 0.6B 或 1B 足够。R122 从本机 290 个 parsed sessions 中抽取 300 个 redacted fragments: 100 个 session、100 个 prompt、100 个 LLM-call。R123 用本地 llama.cpp 3B server 对这 300 个 fragments 做三次 identical repeats，共 900 次请求；结果是 900/900 valid tags，285/300 exact-stable fragments (95.000%)，模型加载约 1,002 ms，请求延迟 p50/p95 为 11/31 ms。这支持 3B syntax、latency 和 repeated-input stability，但不支持 human adequacy。OSDI 版本还需要 R124 人工 adequacy labels；如果论文要声称 0.6B/1B，也必须加入真实 GGUF weights 后重跑。一词标签应该被定位为 lossy navigation frames，而不是 semantic truth。

4.7 评估完整性审计

表 4 把当前证据按 claim gate 汇总。这个表的作用不是装饰，而是防止论文把 smoke evidence 写成系统结论。OSDI reviewer 最可能挑战的点是 C4、C5 和 C6: C4 已经有固定 command-mode suite 证据，但还要证明 scope 不是过窄；C5 要证明用户能更快或更准确地回答 forensic questions；C6 要证明一词标签不只是语法稳定，而是足够可用。因此当前最诚实的定位是 “supported mechanism plus partial systems artifact”。

5 相关工作

Flamegraphs 与 profiling. Brendan Gregg 的 flamegraph 让用户通过宽度定位 hot stacks。Grafana/Pyroscope、Datadog、Sentry 等系统已经把 flamegraph 用于 CPU、内存、profile 和 traces。AgentFlame 借用 folded stack 表示，但 stack frame 来自 agent semantic context 与 system-effect lineage，而不是函数调用栈。

Distributed tracing 与 agent observability. OpenTelemetry/Dapper 风格 tracing 把一次请求表示为

span tree。LangSmith、Langfuse、Phoenix、AgentOps 等工具已经记录 agent、LLM、tool、retrieval、cost 和 latency。SigNoz/Inkeep 公开展示了 multi-agent workflow 的 span-duration flamegraph。这些系统擅长解释单次 workflow 如何执行；AgentFlame 的目标是跨 session 聚合 “哪个语义任务造成哪些系统副作用”。

Token/context profiling. Context 和 token profiling 工具关注 prompt/context window 中的热点。AgentFlame 包含 token flamegraph，但当前只作为 source-local accounting。论文主贡献是 token/semantic/tool/process/effect 共享同一 folded-stack 语法，而不是 token cost 本身。

6 局限与下一步

当前版本仍不到 OSDI weak accept。第一，exact lineage 是最强系统贡献；R114 已在 20 个固定 command-mode Codex 任务上通过，但还没有 full-history exact integration 或跨仓库 benchmark。第二，用户效用没有实验。第三，标签 adequacy 没有人工评估。第四，当前结果来自一个本机仓库，且历史是 observational，不是 paired benchmark。

最短补强路径是：

1. 对 R122 的 300 个 session/prompt/LLM fragments 收集 R124 人工 adequacy labels。
2. 在 R114/R131 traces 上生成 trace tree、span-duration flamegraph、flat summary、nonsemantic stack、semantic stack 五种视图。
3. 用 12–20 个 forensic questions 运行开发者任务 benchmark。
4. 如果要声称更小模型，加入真实 0.6B/1B GGUF weights 并重跑 tagger benchmark。

7 结论

AgentFlame 的核心不是再画一张 agent flamegraph，而是把用户语义意图、LLM/tool 历史和系统副作用放入同一个可折叠、可聚合、可审计的栈模型。当前 full run 说明该模型在真实本地 agent 历史上可运行，并且语义帧能分开 nonsemantic 和 flat baselines 大量混合的系统效应。R114 进一步说明固定 command-mode suite 中 exact semantic-effect lineage 可以做到高精度，并拒绝并发

表 4: 当前 claim gate。Supported 表示已有 committed artifact 支持; partial 表示机制成立但 scope 仍窄; unsupported 表示缺少必要实验。

Claim	当前证据	Verdict	缺口
C1 folded aggregation	167,005 system observations 折叠为 24,295 semantic stacks; sampled artifact verifier 通过。	supported	需要把 full-run artifact packaging 固化成 release command。
C2 one-word tags	2,463 prompt rows 和 90,930 LLM calls 全部满足一词 grammar; 0 final tag failures。	supported	Syntax 不等于 adequacy; 需要人工标签质量评估。
C3 semantic value	Nonsemantic/flat baselines 分别有 90.219% 和 90.770% mixed weight; R131 中 prompt-only 将 system bucket/residual mixed weight 降到 37.687%/7.526%。	supported	需要更多仓库和用户任务证明不是单仓库 workload 特例。
C4 exact lineage	R114 在 20 个固定 codex tasks 上 join 1,273/1,273 in-scope effects, 0 false positives, 0 false negatives, 0/3,170 negative-control effects joined。	fixed-suite supported	需要 full-history integration、跨仓库 replication、更多 agent 类型和 network-effect cases。
C5 user utility	已有 task packet 和 scorer; 尚无 participant responses。	unsupported	需要 trace tree/span flame-graph/flat/semantic 视图的 paired user benchmark。
C6 tag adequacy	3B full run syntax gate 通过; R123 真实 redacted fragments 上 900/900 valid, 285/300 exact-stable, p95 31 ms; 可观察到少量 noisy tags。	partial	需要 R124 人工 adequacy labels; 若要声称 0.6B/1B 还需真实模型 benchmark。

negative controls。但顶会版本必须继续证明 broader/full-history exact lineage、标签 adequacy 和用户任务收益。在当前证据下，最诚实的论文定位是：一个有完整系统化方向和真实 characterization 的语义系统效应 profiler，而不是已经完成的 OSDI artifact。

参考文献

[1] Brendan Gregg. Flame Graphs. <https://www.brendangregg.com/flamegraphs.html>.

[2] Benjamin H. Sigelman et al. Dapper, a Large-Scale Distributed Systems Tracing Infrastructure. Google Technical Report, 2010.

[3] SigNoz. Understanding Flame Graphs for Visualizing Distributed Tracing. <https://signoz.io/blog/flamegraphs/>.

[4] SigNoz. How Inkeep Monitors Their AI Agent Framework with SigNoz. <https://signoz.io/blog/inkeep-ai-agent-monitoring/>.

[5] Datadog. Trace View. https://docs.datadoghq.com/tracing/trace_explorer/trace_view/.

[6] LangChain. LangSmith tracing documentation. <https://docs.langchain.com/langsmith/>.

[7] Langfuse. Observability documentation. <https://langfuse.com/docs/observability>.

[8] Arize Phoenix. Tracing documentation. <https://arize.com/docs/phoenix>.